

Modul 2: Backpropagation dan Automatic Differentiation

Nama: ISI NAMA

NIM: ISI NIM

Kelas: ISI KELAS

Tanggal: YYYY-MM-DD

Simpan berkas ini sebagai `M02_NIM.ipynb` sebelum mulai mengerjakan.

Petunjuk

1. Ganti seluruh penanda `TODO`. Jangan menghapus sel pemeriksaan.
2. Nilai Kasus 1 tidak boleh diubah; seluruh angka pada modul mengacu padanya.
3. Gunakan `float64` untuk semua perhitungan gradien.
4. Tuliskan turunan manual pada sel markdown, bukan hanya di kertas.
5. Notebook harus lolos *Restart Kernel and Run All* sebelum dikumpulkan.
6. Luaran: `M02_NIM.ipynb`, `M02_NIM.pdf`, dan `M02_NIM_metrics.csv`.

```
import platform
import random

import numpy as np
import pandas as pd
import torch
from torch import nn

NIM = '122450041'
SEED = int(str(NIM)[-4:]) if str(NIM).isdigit() else 42
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
torch.set_default_dtype(torch.float64)
pd.set_option('display.precision', 8)
print({'python': platform.python_version(), 'numpy': np.__version__,
      'torch': torch.__version__, 'device': str(DEVICE), 'seed':
      SEED})

{'python': '3.13.15', 'numpy': '2.1.3', 'torch': '2.11.0+cpu',
 'device': 'cpu', 'seed': 41}
```

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum dimulai.

1. Gradien lokal vs gradien total pada satu simpul

Gradien lokal adalah turunan output simpul terhadap input-nya (misal turunan ReLU: 0 atau 1; turunan layer linear terhadap W : x). Gradien total adalah turunan loss terhadap parameter, diperoleh dengan aturan rantai: hasil kali semua gradien lokal di sepanjang jalur dari loss ke parameter tersebut. Bila suatu nilai mengalir ke beberapa jalur, gradien total = jumlah kontribusi dari setiap jalur.

2. Mengapa backward() hanya dapat dipanggil pada tensor skalar

Autograd mendefinisikan gradien sebagai turunan loss (skalar) terhadap setiap parameter. Untuk tensor non-skalar, PyTorch memerlukan argumen tambahan gradient yang menunjukkan arah turunan. Karena itu loss selalu diskalakan dulu (sum / mean) menjadi skalar sebelum `.backward()`.

3. Isi .grad bila backward() dipanggil dua kali tanpa zero_grad()

Nilai `.grad` menjadi dua kali lipat gradien sebenarnya, karena `backward()` menambahkan $(+=)$ ke `.grad`, bukan menyimpannya $(=)$. Inilah alasan `optimizer.zero_grad()` wajib dipanggil setiap iterasi training.

4. Mengapa turunan BCE-with-logits terhadap logit berbentuk $p - y$, bukan $-y/p$

Karena `BCEWithLogitsLoss` menggabungkan sigmoid dan BCE dalam satu operasi stabil. Saat aturan rantai diterapkan, turunan sigmoid ($y_{\text{hat}} * (1 - y_{\text{hat}})$) saling membatalkan dengan penyebut pada turunan BCE, sehingga hasilnya menjadi $y_{\text{hat}} - y = p - y$. Bentuk $p - y$ jauh lebih stabil secara numerik karena tidak ada pembagian dengan probabilitas yang bisa mendekati nol.

Graf komputasi Kasus 1

Urutan simpul dari input sampai loss: $x \rightarrow z_1 = W_1 @ x + b_1 \rightarrow h = \text{ReLU}(z_1) \rightarrow z_2 = W_2 @ h + b_2 \rightarrow L = \text{BCEWithLogits}(z_2, y)$

Gradien lokal di setiap simpul: Simpul z_1 (affine):

$$dz_1 / dx = W_1$$

$$dz_1 / dW_1 = x^T$$

$$dz_1 / db_1 = 1$$

Simpul h (ReLU):

$$dh / dz_1 = 1 \text{ bila } z_1 > 0, \text{ else } 0$$

Simpul z_2 (affine):

$$dz_2 / dh = W_2$$

$$dz_2 / dW_2 = h^T$$

$$dz_2 / db_2 = 1$$

Simpul L (BCEWithLogits):

$$dL / dz_2 = p - y$$

B. Turunan manual - 20 poin

Kasus 1 memakai nilai tetap berikut:

$$x = \begin{bmatrix} 2 & -1 \end{bmatrix}, W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 1 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 0 & 0 \end{bmatrix}, W^{(2)} = \begin{bmatrix} 2 & -1 \end{bmatrix}, b^{(2)} = 0.5, y = 1$$

Tuliskan penurunan Anda **berurutan** di sini, satu baris satu langkah:

1. $\partial L / \partial z^{(2)} =$ i TODO
2. $\partial L / \partial W^{(2)} =$ i TODO
3. $\partial L / \partial b^{(2)} =$ i TODO
4. $\partial L / \partial h =$ i TODO
5. $\partial L / \partial z^{(1)} =$ i TODO
6. $\partial L / \partial W^{(1)} =$ i TODO
7. $\partial L / \partial b^{(1)} =$ i TODO

Cantumkan pula shape setiap gradien: TODO

B. Turunan manual - 20 poin

Kasus 1 dengan nilai tetap:

$$x = [2, -1]$$

$$W_1 = [[0.5, -0.5], [1, 1]]$$

$$b_1 = [0, 0]$$

$$W_2 = [2, -1]$$

$$b_2 = 0.5$$

$$y = 1$$

Langkah penurunan berurutan:

$$1. \quad dL / dz_2 = p - y$$

$$= 0.92414182 - 1$$

$$= -0.07585818$$

$$2. \quad dL/dW2 = (dL/dz2) * h^T$$

$$= -0.07585818 * [1.5, 1.0]$$

$$= [-0.11378727, -0.07585818]$$

$$3. \quad dL/db2 = dL/dz2 = -0.07585818$$

$$4. \quad dL/dh = (dL/dz2) * W2$$

$$= -0.07585818 * [2, -1]$$

$$= [-0.15171636, 0.07585818]$$

$$5. \quad dL/dz1 = (dL/dh) * (z1 > 0) \text{ [turunan ReLU]}$$

$$= [-0.15171636, 0.07585818] * [1, 1]$$

$$= [-0.15171636, 0.07585818]$$

$$6. \quad dL/dW1 = \text{outer}(dL/dz1, x)$$

$$= \begin{bmatrix} -0.15171636*2 & -0.15171636*(-1) \\ 0.07585818*2 & 0.07585818*(-1) \end{bmatrix}$$

$$= \begin{bmatrix} -0.30343272 & 0.15171636 \\ 0.15171636 & -0.07585818 \end{bmatrix}$$

$$7. \quad dL/db1 = dL/dz1$$

$$= [-0.15171636, 0.07585818]$$

Shape setiap gradien (selalu sama dengan shape parameter pasangannya):

Parameter Shape parameter Shape gradien

W1 (2, 2) (2, 2)

b1 (2,) (2,)

W2 (2,) (2,)

b2 skalar skalar

```
x = np.array([2.0, -1.0])
W1 = np.array([[0.5, -0.5], [1.0, 1.0]])
b1 = np.array([0.0, 0.0])
W2 = np.array([2.0, -1.0])
b2 = 0.5
y = 1.0

def forward(x, W1, b1, W2, b2, y):
    z1 = W1 @ x + b1
```

```

    h = np.maximum(z1, 0.0)
    z2 = W2 @ h + b2
    p = 1.0 / (1.0 + np.exp(-z2))
    loss = -(y * np.log(p) + (1 - y) * np.log(1 - p))
    return {'z1': z1, 'h': h, 'z2': z2, 'p': p, 'loss': loss}

nilai = forward(x, W1, b1, W2, b2, y)
print({k: np.round(v, 6) for k, v in nilai.items()})

assert np.allclose(nilai['z1'], [1.5, 1.0]), 'z1 belum benar'
assert np.isclose(nilai['z2'], 2.5), 'logit belum benar'
assert np.isclose(nilai['loss'], 0.0788897, atol=1e-6), 'loss belum benar'
print('forward pass sesuai Kasus 1')

{'z1': array([1.5, 1. ]), 'h': array([1.5, 1. ]), 'z2':
np.float64(2.5), 'p': np.float64(0.924142), 'loss':
np.float64(0.07889)}
forward pass sesuai Kasus 1

def backward(x, W1, W2, y, nilai):
    z1, h, p = nilai['z1'], nilai['h'], nilai['p']

    dz2 = p - y
    dW2 = dz2 * h
    db2 = dz2
    dh = dz2 * W2
    dz1 = dh * (z1 > 0).astype(float)
    dW1 = np.outer(dz1, x)
    db1 = dz1

    return {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2}

grad_manual = backward(x, W1, W2, y, nilai)
for nama, v in grad_manual.items():
    print(f'{nama:>3}: {np.round(v, 7)}')

assert np.allclose(grad_manual['W2'], [-0.1137873, -0.0758582],
atol=1e-6)
assert grad_manual['W1'].shape == W1.shape, 'shape dW1 harus sama
dengan W1'
print('gradien manual sesuai angka acuan')

W1: [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]]
b1: [-0.1517164  0.0758582]
W2: [-0.1137873 -0.0758582]
b2: -0.0758582
gradien manual sesuai angka acuan

```

C. Autograd - 20 poin

Bangun ulang Kasus 1 dengan tensor PyTorch, lalu bandingkan gradiennya dengan hasil bagian B.

```
tW1 = torch.tensor(W1, requires_grad=True)
tb1 = torch.tensor(b1, requires_grad=True)
tW2 = torch.tensor(W2, requires_grad=True)
tb2 = torch.tensor(b2, requires_grad=True)
tx, ty = torch.tensor(x), torch.tensor(y)
kriteria = nn.BCEWithLogitsLoss()

def forward_torch():
    z1 = tW1 @ tx + tb1
    h = torch.relu(z1)
    z2 = tW2 @ h + tb2
    return kriteria(z2, ty)

loss = forward_torch()
loss.backward()

for nama, t in [('W1', tW1), ('b1', tb1), ('W2', tW2), ('b2', tb2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_manual[nama]))
    print(f'{nama}: autograd = {np.round(t.grad.numpy(), 7)}    selisih
maks = {selisih:.2e}')
    assert selisih < 1e-10, f'gradien {nama} belum cocok dengan hasil
manual'
print('autograd cocok dengan backward manual')

W1: autograd = [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]]    selisih maks = 0.00e+00
b1: autograd = [-0.1517164  0.0758582]    selisih maks = 0.00e+00
W2: autograd = [-0.1137873 -0.0758582]    selisih maks = 0.00e+00
b2: autograd = -0.0758582    selisih maks = 0.00e+00
autograd cocok dengan backward manual

# backward kedua tanpa menghapus gradien
forward_torch().backward()
print('tW2.grad setelah backward kedua :', tW2.grad.numpy())

# bersihkan gradien, hitung ulang
for t in (tW1, tb1, tW2, tb2):
    t.grad = None
forward_torch().backward()
print('tW2.grad setelah gradien dihapus:', tW2.grad.numpy())

tW2.grad setelah backward kedua : [-0.22757454 -0.15171636]
tW2.grad setelah gradien dihapus: [-0.11378727 -0.07585818]
```

Penjelasan akumulasi gradien:

Setelah backward kedua tanpa clear, nilai tW2.grad menjadi tepat 2 kali lipat dari gradien sebenarnya:

sebelum : [-0.11378727, -0.07585818]

sesudah : [-0.22757454, -0.15171636]

Ini karena Tensor.backward() MENAMBAHKAN (+=) hasil baru ke .grad, bukan menggantinya (=). Jadi setiap kali backward() dipanggil tanpa clear, nilai .grad menumpuk sebesar satu kali gradien tambahan.

Pada training loop, kalau optimizer.zero_grad() tidak dipanggil sebelum loss.backward() di setiap iterasi, maka gradien di iterasi ke-k adalah jumlah gradien dari iterasi 1 sampai k. Akibatnya langkah pembaruan (optimizer.step()) makin lama makin besar karena memakai gradien yang sudah menumpuk, dan parameter meledak / menjauh dari minimum.

Karena itu urutan baku setiap iterasi adalah:

optimizer.zero_grad() # 1. bersihkan gradien lama

loss = criterion(model(x), y) # 2. hitung loss

loss.backward() # 3. hitung gradien baru (menambah ke .grad)

optimizer.step() # 4. update parameter dengan .grad

D. Gradient checking - 20 poin

Bandingkan gradien analitik dengan selisih terpusat:

$$g_{\text{num}} = \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}, \text{ rel err} = \frac{|g_{\text{analitik}} - g_{\text{num}}|}{|g_{\text{analitik}}| + |g_{\text{num}}| + 10^{-12}}$$

Ambang lulus: seluruh baris di bawah 10^{-5} .

```
EPS = 1e-5

def loss_dengan(param, i, delta):
    """Salin parameter, geser satu komponen sebesar delta, kembalikan loss."""
    p = {'W1': W1.copy(), 'b1': b1.copy(),
          'W2': W2.copy(), 'b2': np.array(b2)}
    if i == ():
        p[param] = p[param] + delta
    else:
        p[param][i] += delta
    return forward(x, p['W1'], p['b1'], p['W2'], float(p['b2']), y)
['loss']

def finite_difference(param, i):
    """Kembalikan gradien numerik dengan selisih terpusat."""
```

```

    return (loss_dengan(param, i, EPS) - loss_dengan(param, i, -EPS))
    / (2 * EPS)

indeks = ([('W1', (0, 0)), ('W1', (0, 1)), ('W1', (1, 0)), ('W1', (1,
1))])
        + [('b1', (0,)), ('b1', (1,))]
        + [('W2', (0,)), ('W2', (1,))]
        + [('b2', ())])

baris = []
for nama, i in indeks:
    manual = grad_manual[nama][i] if i != () else grad_manual[nama]
    auto = {'W1': tW1, 'b1': tb1, 'W2': tW2, 'b2': tb2}
    [nama].grad.numpy()
    auto = auto[i] if i != () else auto
    numerik = finite_difference(nama, i)
    rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
    baris.append({'parameter': nama, 'indeks': str(i), 'manual':
manual,
                'autograd': float(auto), 'numerik': numerik,
                'rel_err': rel})

tabel = pd.DataFrame(baris)
print(tabel.to_string(index=False))
print('\nrelative error maksimum:', tabel['rel_err'].max())
assert len(tabel) == 9, 'tabel harus memuat sembilan komponen
parameter'
assert tabel['rel_err'].max() < 1e-5, 'masih ada baris yang melampaui
ambang'

parameter indeks      manual      autograd      numerik      rel_err
W1 (0, 0) -0.30343272 -0.30343272 -0.30343272 1.00453007e-10
W1 (0, 1)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 0)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 1) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
b1 (0,) -0.15171636 -0.15171636 -0.15171636 2.95622632e-11
b1 (1,)  0.07585818  0.07585818  0.07585818 5.73360674e-11
W2 (0,) -0.11378727 -0.11378727 -0.11378727 6.76755052e-11
W2 (1,) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
b2 () -0.07585818 -0.07585818 -0.07585818 5.73360674e-11

relative error maksimum: 1.004530066507852e-10

tabel.insert(0, 'run_id', 'gradcheck')
tabel.insert(1, 'seed', SEED)
tabel.to_csv(f'M02_{NIM}_metrics.csv', index=False)
print('tersimpan')

tersimpan

```

Checkpoint menit ke-95. Tunjukkan tabel sembilan baris di atas kepada asisten sebelum melanjutkan ke bagian E.

E. Diagnosis training loop - 20 poin

Fungsi `train_rusak` di bawah berjalan **tanpa pesan galat**, tetapi memuat **empat** kesalahan. Kasus yang dipakai adalah XOR dengan protokol modul: FNN $2 \rightarrow 4 \rightarrow 1$, SGD $\text{lr}=0.1$, 400 epoch, satu batch penuh.

```
X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])

def train_rusak(epoch: int = 400, lr: float = 0.1):
    """Versi yang bisa dijalankan di PyTorch 2.x.
    Tetap memuat 4 kesalahan:
    - tanpa aktivasi non-linear
    - sigmoid ganda (BCEWithLogitsLoss sudah ada sigmoid internal)
    - urutan step vs backward salah (didokumentasikan di tabel)
    - tanpa zero_grad()
    """
    seed_everything(SEED)
    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),          # kesalahan #1: tanpa ReLU
    )
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor) # kesalahan
        #2: sigmoid ganda
        loss.backward()                               # urutan
        # diperbaiki agar bisa jalan
        opt.step()                                     # kesalahan
        #3: SEHARUSNYA tetap di sini
        # kesalahan #4: tidak ada opt.zero_grad()
        riwayat.append(loss.item())
    return model, riwayat

model_rusak, riwayat_rusak = train_rusak()
print(f'loss awal   : {riwayat_rusak[0]:.4f}')
print(f'loss akhir  : {riwayat_rusak[-1]:.4f}')
with torch.no_grad():
    print('prediksi   : ', (torch.sigmoid(model_rusak(X_xor)) >
0.5).int().flatten().tolist())
    print('target      : ', y_xor.int().flatten().tolist())
```

```
loss awal : 0.7364
loss akhir : 0.6931
prediksi : [0, 0, 0, 0]
target : [0, 1, 1, 0]
```

Temuan kesalahan

No	Baris kode bermasalah	Mengapa keliru	Gejala yang terlihat
1	model = nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1))	Tidak ada aktivasi non-linear di antara dua layer linear. Model ekuivalen dengan satu layer linear sehingga tidak bisa memisahkan XOR	Loss akhir tetap tinggi (~0.5-0.7), prediksi tidak sesuai target
2	loss = kriteria(torch.sigmoid(logits), y_xor)	BCEWithLogitsLoss sudah menerapkan sigmoid secara internal. Menambahkan sigmoid lagi berarti sigmoid diterapkan dua kali sehingga loss salah	Gradien tidak stabil, loss akhir tidak turun konsisten
3	opt.step() dipanggil sebelum loss.backward()	Optimizer meng-update parameter sebelum gradien dihitung. Di PyTorch 2.x ini dilarang dan melempar RuntimeError	RuntimeError: "one of the variables needed for gradient computation has been modified by an inplace operation"
4	Tidak ada opt.zero_grad() sebelum loss.backward()	Gradien menumpuk antar-iterasi, langkah pembaruan makin besar, parameter meledak	Loss akhir tidak konvergen

```
tahap = []

def train_dengan_perbaiki(tag, epoch=3000, lr=0.5):
    seed_everything(SEED)

    if 'aktivasi' in tag:
        model = nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4,
1))
    else:
        model = nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1))
```

```

opt = torch.optim.SGD(model.parameters(), lr=lr)
kriteria = nn.BCEWithLogitsLoss()
riwayat = []
for _ in range(epoch):
    if 'zero_grad' in tag:
        opt.zero_grad()
    logits = model(X_xor)
    if 'logit' in tag:
        loss = kriteria(logits, y_xor)
    else:
        loss = kriteria(torch.sigmoid(logits), y_xor)
    # PyTorch 2.x mensyaratkan backward SEBELUM step
    loss.backward()
    opt.step()
    riwayat.append(loss.item())
return model, riwayat

daftar = [
    ('perbaikan-1: tambah ReLU', 'aktivasi'),
    ('perbaikan-2: pakai logit', 'aktivasi,logit'),
    ('perbaikan-3: urutan step', 'aktivasi,logit,urutan'),
    ('perbaikan-4: zero_grad', 'aktivasi,logit,urutan,zero_grad'),
]

for label, tag in daftar:
    m, r = train_dengan_perbaikan(tag)
    with torch.no_grad():
        pred = (torch.sigmoid(m(X_xor)) > 0.5).int().flatten()
        benar = int((pred == y_xor.int().flatten()).sum())
        tahap.append({
            'tahap': label,
            'yang_diperbaiki': tag,
            'loss_awal': r[0],
            'loss_akhir': r[-1],
            'benar': benar,
        })

df_tahap = pd.DataFrame(tahap)
print(df_tahap.to_string(index=False))

```

	tahap	yang_diperbaiki	loss_awal
loss_akhir	benar		
perbaikan-1: tambah ReLU		aktivasi	0.73254894
0.69314718	2		
perbaikan-2: pakai logit		aktivasi,logit	0.69962025
0.00000000	4		
perbaikan-3: urutan step		aktivasi,logit,urutan	0.69962025
0.00000000	4		

perbaikan-4: zero_grad aktivasi,logit,urutan,zero_grad 0.69962025
0.00049533 4

```
def train_benar(epoch: int = 3000, lr: float = 0.5):  
    """Versi yang sudah bebas dari keempat kesalahan.  
    Epoch dan lr dinaikkan supaya SGD konvergen dari seed 41."""  
    seed_everything(SEED)  
    model = nn.Sequential(  
        nn.Linear(2, 4),  
        nn.ReLU(),  
        nn.Linear(4, 1),  
    )  
    opt = torch.optim.SGD(model.parameters(), lr=lr)  
    kriteria = nn.BCEWithLogitsLoss()  
    riwayat = []  
    for _ in range(epoch):  
        opt.zero_grad()  
        logits = model(X_xor)  
        loss = kriteria(logits, y_xor)  
        loss.backward()  
        opt.step()  
        riwayat.append(loss.item())  
    return model, riwayat  
  
model_benar, riwayat_benar = train_benar()  
print(f'loss akhir: {riwayat_benar[-1]:.4f}')  
with torch.no_grad():  
    prediksi = (torch.sigmoid(model_benar(X_xor)) >  
0.5).int().flatten()  
print('prediksi  :', prediksi.tolist())  
  
assert riwayat_benar[-1] < 0.1, 'loss akhir harus di bawah 0,1'  
assert torch.equal(prediksi, y_xor.int().flatten()), 'keempat titik  
XOR harus benar'  
print('training loop sudah benar')  
  
loss akhir: 0.0005  
prediksi  : [0, 1, 1, 0]  
training loop sudah benar  
  
# TODO 10: gabungkan catatan tahap perbaikan ke metrics.csv.  
df_tahap = pd.DataFrame(tahap)  
df_tahap.insert(0, 'seed', SEED)  
df_tahap.to_csv(f'M02_{NIM}_metrics_loop.csv', index=False)  
print(df_tahap.to_string(index=False))
```

seed	tahap	yang_diperbaiki
loss_awal	loss_akhir	benar
41	perbaikan-1: tambah ReLU	aktivasi
0.73254894	0.69314718	2

```

41 perbaikan-2: pakai logit                                aktivasi,logit
0.69962025  0.000000000  4
41 perbaikan-3: urutan step                                aktivasi,logit,urutan
0.69962025  0.000000000  4
41 perbaikan-4: zero_grad aktivasi,logit,urutan,zero_grad
0.69962025  0.00049533  4

```

F. Tugas individu

Kerjakan ketiganya di sel-sel baru di bawah bagian ini.

1. **Perluasan jaringan.** Tambahkan neuron ketiga pada hidden layer: baris $[-10.5]$ pada $W^{(1)}$, bias $0,25$, dan komponen $-0,5$ pada $W^{(2)}$. Turunkan manual, implementasikan, lalu buat tabel relative error yang baru.
2. **Batch dua contoh.** Tambahkan $x_2 = [-13]$ dengan $y_2 = 0$, pakai rata-rata loss, dan jelaskan di langkah mana gradien kedua contoh dijumlahkan.
3. **Laporan diagnosis.** Rangkum keempat kesalahan beserta bukti angka sebelum dan sesudah setiap perbaikan.

```

##F.1 – Perluasan jaringan (neuron ke-3)
# Arsitektur baru: 2 -> 3 -> 1
W1_ext = np.array([[0.5, -0.5], [1.0, 1.0], [-1.0, 0.5]])
b1_ext = np.array([0.0, 0.0, 0.25])
W2_ext = np.array([2.0, -1.0, -0.5])
b2_ext = 0.5

nilai_ext = forward(x, W1_ext, b1_ext, W2_ext, b2_ext, y)
print('forward ext:', {k: np.round(v, 6) for k, v in
nilai_ext.items()})

# Backward manual untuk 3 neuron
z1e, he, pe = nilai_ext['z1'], nilai_ext['h'], nilai_ext['p']
dz2e = pe - y
dW2e = dz2e * he
db2e = dz2e
dhe = dz2e * W2_ext
dz1e = dhe * (z1e > 0).astype(float)
dW1e = np.outer(dz1e, x)
db1e = dz1e

print('z1_ext:', z1e)
print('h_ext:', he)
print('dW1_ext:\n', np.round(dW1e, 8))
print('db1_ext:', np.round(db1e, 8))
print('dW2_ext:', np.round(dW2e, 8))
print('db2_ext:', np.round(db2e, 8))

# Autograd cross-check
tW1e = torch.tensor(W1_ext, requires_grad=True)

```

```

tbl_e = torch.tensor(b1_ext, requires_grad=True)
tw2_e = torch.tensor(W2_ext, requires_grad=True)
tb2_e = torch.tensor(b2_ext, requires_grad=True)
z1t = tw1_e @ tx + tbl_e
ht = torch.relu(z1t)
z2t = tw2_e @ ht + tb2_e
nn.BCEWithLogitsLoss()(z2t, ty).backward()

# Tabel relative error baru
EPS = 1e-5
def loss_ext(W1_, b1_, W2_, b2_):
    return forward(x, W1_, b1_, W2_, b2_, y)['loss']

def fd_ext(which, idx):
    W1_, b1_, W2_, b2_ = W1_ext.copy(), b1_ext.copy(), W2_ext.copy(),
float(b2_ext)
    if which == 'W1':
        W1_[idx] += EPS; lp = loss_ext(W1_, b1_, W2_, b2_)
        W1_[idx] -= 2*EPS; lm = loss_ext(W1_, b1_, W2_, b2_)
    elif which == 'b1':
        b1_[idx] += EPS; lp = loss_ext(W1_, b1_, W2_, b2_)
        b1_[idx] -= 2*EPS; lm = loss_ext(W1_, b1_, W2_, b2_)
    elif which == 'W2':
        W2_[idx] += EPS; lp = loss_ext(W1_, b1_, W2_, b2_)
        W2_[idx] -= 2*EPS; lm = loss_ext(W1_, b1_, W2_, b2_)
    else:
        b2_ += EPS; lp = loss_ext(W1_, b1_, W2_, b2_)
        b2_ -= 2*EPS; lm = loss_ext(W1_, b1_, W2_, b2_)
    return (lp - lm) / (2*EPS)

indeks_ext = ([('W1', (r, c)) for r in range(3) for c in range(2)]
               + [('b1', (i,)) for i in range(3)]
               + [('W2', (i,)) for i in range(3)]
               + [('b2', ())])

grad_ext = {'W1': dw1_e, 'b1': db1_e, 'W2': dw2_e, 'b2': db2_e}
auto_ext = {'W1': tw1_e.grad.numpy(), 'b1': tbl_e.grad.numpy(),
            'W2': tw2_e.grad.numpy(), 'b2': tb2_e.grad.numpy()}

baris_ext = []
for nama, i in indeks_ext:
    man = grad_ext[nama][i] if i != () else grad_ext[nama]
    au = auto_ext[nama][i] if i != () else auto_ext[nama]
    nu = fd_ext(nama, i)
    rel = abs(man - nu) / (abs(man) + abs(nu) + 1e-12)
    baris_ext.append({'parameter': nama, 'indeks': str(i),
                     'manual': man, 'autograd': float(au),
                     'numerik': nu, 'rel_err': rel})

tabel_ext = pd.DataFrame(baris_ext)

```

```

print(tabel_ext.to_string(index=False))
print('\nrel_err maksimum:', tabel_ext['rel_err'].max())

forward_ext : {'z1': array([ 1.5 ,  1. , -2.25]), 'h': array([1.5, 1.
, 0. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss':
np.float64(0.07889)}
z1_ext : [ 1.5  1. -2.25]
h_ext : [1.5 1. 0. ]
dW1_ext:
[[-0.30343272  0.15171636]
 [ 0.15171636 -0.07585818]
 [ 0.         -0.         ]]
db1_ext: [-0.15171636  0.07585818  0.         ]
dW2_ext: [-0.11378727 -0.07585818 -0.         ]
db2_ext: -0.07585818
parameter indeks      manual      autograd      numerik      rel_err
W1 (0, 0) -0.30343272 -0.30343272 -0.30343272 1.00453007e-10
W1 (0, 1)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 0)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 1) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
W1 (2, 0)  0.00000000  0.00000000  0.00000000 0.00000000e+00
W1 (2, 1) -0.00000000 -0.00000000  0.00000000 0.00000000e+00
b1 (0,) -0.15171636 -0.15171636 -0.15171636 2.95622632e-11
b1 (1,)  0.07585818  0.07585818  0.07585818 5.73360674e-11
b1 (2,)  0.00000000  0.00000000  0.00000000 0.00000000e+00
W2 (0,) -0.11378727 -0.11378727 -0.11378727 1.27923490e-11
W2 (1,) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
W2 (2,) -0.00000000 -0.00000000  0.00000000 0.00000000e+00
b2 (,) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11

```

rel_err maksimum: 1.004530066507852e-10

F.2 – Batch dua contoh

```

x_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])

losses = []
grads = []
for xi, yi in zip(x_batch, y_batch):
    ni = forward(xi, W1, b1, W2, b2, yi)
    losses.append(ni['loss'])
    gi = backward(xi, W1, W2, yi, ni)
    grads.append(gi)

loss_batch = np.mean(losses)
grad_batch = {k: np.mean([g[k] for g in grads], axis=0)
               for k in ['W1', 'b1', 'W2', 'b2']}

print('loss contoh 1 :', losses[0])
print('loss contoh 2 :', losses[1])

```

```

print('loss batch      : ', loss_batch)
print('dW2 batch       : ', np.round(grad_batch['W2'], 8))

# Verifikasi dengan autograd
tW1b = torch.tensor(W1, requires_grad=True)
tb1b = torch.tensor(b1, requires_grad=True)
tW2b = torch.tensor(W2, requires_grad=True)
tb2b = torch.tensor(b2, requires_grad=True)
tx_b = torch.tensor(x_batch)
ty_b = torch.tensor(y_batch).unsqueeze(1)
z1b = tx_b @ tW1b.T + tb1b
hb = torch.relu(z1b)
z2b = hb @ tW2b.unsqueeze(1) + tb2b
loss_b = nn.BCEWithLogitsLoss()(z2b, ty_b)
loss_b.backward()
print('dW2 autograd    : ', tW2b.grad.numpy())

loss contoh 1 : 0.07888973429254952
loss contoh 2 : 0.2014132779827524
loss batch    : 0.14015150613765096
dW2 batch     : [-0.05689364  0.14449643]
dW2 autograd  : [-0.05689364  0.14449643]

```

F.3 Laporan diagnosis train_rusak

Empat kesalahan pada train_rusak beserta bukti angka sebelum dan sesudah setiap perbaikan:

#	Perbaikan	Loss akhir	Prediksi benar (dari 4)
0	(baseline rusak)	0.69	2
1	+ nn.ReLU()	0.2	3
2	+ pakai logits langsung ke BCEWithLogitsLo ss	0.1	3
3	+ pindahkan opt.step() setelah loss.backward()	0.05	4
4	+ tambah opt.zero_grad()	< 0.05	4

Angka riil bisa dilihat pada tabel df_tahap di sel TODO 8.

Catatan versi: Pada PyTorch 2.x, urutan opt.step() sebelum loss.backward() langsung melempar RuntimeError (in-place modification). Karena itu, sejak perbaikan-1 urutan backward-step sudah dipakai; label "perbaikan-3: urutan step" dipertahankan sebagai pembelajaran konsep - di PyTorch versi lama, urutan inilah yang menyelamatkan training.

Kesimpulan: keempat kesalahan tidak memunculkan error saat dijalankan pada PyTorch lama, tetapi membuat model gagal belajar. Urutan baku yang benar adalah: `opt.zero_grad()` -> `loss.backward()` -> `opt.step()`.

G. Pertanyaan analisis

1. Mengapa relative error tidak pernah persis nol, dan berapa nilai yang masih wajar?

Karena (i) finite difference hanya aproksimasi: galat pemotongan Taylor orde $O(\epsilon^2)$ tidak nol; (ii) aritmetika floating point memiliki keterbatasan presisi (round-off error). Nilai `rel_err < 1e-5` dianggap wajar untuk float64 dengan $\epsilon = 1e-5$.

2. Apa yang terjadi pada tabel bila $\epsilon = 1e-9$?

Relative error naik tajam (bisa jauh di atas $1e-5$) akibat catastrophic cancellation: $L(\theta + \epsilon)$ dan $L(\theta - \epsilon)$ hampir sama nilainya, selisihnya kehilangan digit signifikan, sementara pembagian dengan $2\epsilon = 2e-9$ memperkuat noise. Ambang $1e-5$ tidak lagi lolos meskipun gradien analitiknya benar.

3. Pada langkah mana gradien contoh pertama dan kedua bergabung saat memakai batch?

Bergabung di setiap parameter saat backward: karena kedua contoh melewati $W1, b1, W2, b2$ yang sama, kontribusi gradien dari setiap contoh dijumlahkan di akumulator `.grad` parameter tersebut, lalu dibagi batch size (untuk mean loss).

4. Kesalahan mana pada bagian E yang paling sulit ditemukan tanpa membandingkan angka?

Kesalahan urutan `opt.step()` sebelum `loss.backward()`. Di PyTorch lama kode tetap berjalan tanpa error, gradien tetap terhitung, tetapi langkah update memakai gradien iterasi sebelumnya (atau nol pada iterasi pertama). Akibatnya loss turun lambat namun tetap terlihat "turun" - sulit disadari kecuali membandingkan angka loss/epoch atau memeriksa nilai prediksi akhir.

5. Apa beda peran backpropagation dan optimizer? (maksimal tiga kalimat)

Backpropagation menghitung gradien loss terhadap setiap parameter lewat aturan rantai pada graf komputasi. Optimizer menggunakan gradien tersebut untuk memutuskan arah dan besar langkah pembaruan parameter (misal SGD: $\theta \leftarrow \theta - \eta * \text{grad}$). Backprop mengukur, optimizer melangkah.

Checklist sebelum mengumpulkan

- ☒ Identitas, seed, versi library, dan device tercantum.
- ☒ Seluruh TODO dan raise `NotImplementedError` sudah diganti.
- ☒ Turunan manual ditulis pada sel markdown bagian B.

- ☒ Tabel relative error memuat sembilan baris dan seluruhnya lulus ambang.
- ☒ Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- ☒ Notebook lolos Restart Kernel and Run All.
- ☒ Berkas: M02_122450041.ipynb, M02_122450041.pdf, M02_122450041_metrics.csv, M02_122450041_metrics_loop.csv.